

TABEL PENGUJIAN

===PROGRAM 5===

PYTHON :

No	Input Titik U	Hasil yang Diharapkan	Hasil Program	Status
1	(1, 2, 3)	Masuk ke Cluster A	Titik U masuk ke Cluster A	Berhasil
2	(1, -3, 5)	Masuk ke Cluster B	Titik U masuk ke Cluster B	Berhasil
3	(-1, 3, -1)	Masuk ke Cluster C	Titik U masuk ke Cluster C	Berhasil

```

import math

# Titik pusat cluster
A = (2, 1, 3)
B = (1, -4, 6)
C = (-2, 3, -2)

# Titik yang akan diklasifikasikan
U = (1, 2, 3)

# Fungsi menghitung jarak Euclidean
def jarak(p, q):
    return math.sqrt((p[0] - q[0])**2 +
                     (p[1] - q[1])**2 +
                     (p[2] - q[2])**2)

# Hitung jarak ke masing-masing cluster
dA = jarak(U, A)
dB = jarak(U, B)
dC = jarak(U, C)

# Menentukan cluster terdekat
if dA <= dB and dA <= dC:
    cluster = "A"
elif dB <= dA and dB <= dC:
    cluster = "B"
else:
    cluster = "C"

# Output hasil
print("Titik U =", U)
print("Jarak ke Cluster A =", round(dA, 3))
print("Jarak ke Cluster B =", round(dB, 3))
print("Jarak ke Cluster C =", round(dC, 3))
print("Titik U masuk ke Cluster", cluster)

... Titik U = (1, 2, 3)
Jarak ke Cluster A = 1.414
Jarak ke Cluster B = 6.708
Jarak ke Cluster C = 5.916
Titik U masuk ke Cluster A

```

```

import math

# Titik pusat cluster
A = (2, 1, 3)
B = (1, -4, 6)
C = (-2, 3, -2)

# Titik yang akan diklasifikasikan
U = (1, -3, 5)

# Fungsi menghitung jarak Euclidean
def jarak(p, q):
    return math.sqrt((p[0] - q[0])**2 +
                     (p[1] - q[1])**2 +
                     (p[2] - q[2])**2)

# Hitung jarak ke masing-masing cluster
dA = jarak(U, A)
dB = jarak(U, B)
dC = jarak(U, C)

# Menentukan cluster terdekat
if dA <= dB and dA <= dC:
    cluster = "A"
elif dB <= dA and dB <= dC:
    cluster = "B"
else:
    cluster = "C"

# Output hasil
print("Titik U =", U)
print("Jarak ke Cluster A =", round(dA, 3))
print("Jarak ke Cluster B =", round(dB, 3))
print("Jarak ke Cluster C =", round(dC, 3))
print("Titik U masuk ke Cluster", cluster)

... Titik U = (1, -3, 5)
Jarak ke Cluster A = 4.583
Jarak ke Cluster B = 1.414
Jarak ke Cluster C = 9.695
Titik U masuk ke Cluster B

```

```

▶ import math

# Titik pusat cluster
A = (2, 1, 3)
B = (1, -4, 6)
C = (-2, 3, -2)

# Titik yang akan diklasifikasikan
U = (-1, 3, -1)

# Fungsi menghitung jarak Euclidean
def jarak(p, q):
    return math.sqrt((p[0] - q[0])**2 +
                     (p[1] - q[1])**2 +
                     (p[2] - q[2])**2)

# Hitung jarak ke masing-masing cluster
dA = jarak(U, A)
dB = jarak(U, B)
dC = jarak(U, C)

# Menentukan cluster terdekat
if dA <= dB and dA <= dC:
    cluster = "A"
elif dB <= dA and dB <= dC:
    cluster = "B"
else:
    cluster = "C"

# Output hasil
print("Titik U =", U)
print("Jarak ke Cluster A =", round(dA, 3))
print("Jarak ke Cluster B =", round(dB, 3))
print("Jarak ke Cluster C =", round(dC, 3))
print("Titik U masuk ke Cluster", cluster)

... Titik U = (-1, 3, -1)
Jarak ke Cluster A = 5.385
Jarak ke Cluster B = 10.1
Jarak ke Cluster C = 1.414
Titik U masuk ke Cluster C

```

R :

No	Input Titik U	Hasil yang Diharapkan	Hasil Program	Status
1	(1, 2, 3)	Masuk ke Cluster A	Titik U masuk ke Cluster A	Berhasil
2	(1, -3, 5)	Masuk ke Cluster B	Titik U masuk ke Cluster B	Berhasil
3	(-1, 3, -1)	Masuk ke Cluster C	Titik U masuk ke Cluster C	Berhasil

```
R • R 4.5.1 • ~/
> # Titik pusat cluster
> A <- c(2, 1, 3)
> B <- c(1, -4, 6)
> C <- c(-2, 3, -2)
> # Titik yang akan diklasifikasikan
> U <- c(1, 2, 3)
> # Fungsi menghitung jarak
> jarak <- function(p, q) {
+   sqrt(sum((p - q)^2))
+ }
> # Hitung jarak
> dA <- jarak(U, A)
> dB <- jarak(U, B)
> dC <- jarak(U, C)
> # Tentukan cluster
> if (dA <= dB && dA <= dC) {
+   cluster <- "A"
+ } else if (dB <= dA && dB <= dC) {
+   cluster <- "B"
+ } else {
+   cluster <- "C"
+ }
> cat("Titik U =", U, "\n")
Titik U = 1 2 3
> cat("Jarak ke Cluster A =", round(dA, 3), "\n")
Jarak ke Cluster A = 1.414
> cat("Jarak ke Cluster B =", round(dB, 3), "\n")
Jarak ke Cluster B = 6.708
> cat("Jarak ke Cluster C =", round(dC, 3), "\n")
Jarak ke Cluster C = 5.916
> cat("Titik U masuk ke Cluster", cluster, "\n")
Titik U masuk ke Cluster A
> |
```

```
> # Titik pusat cluster
> A <- c(2, 1, 3)
> B <- c(1, -4, 6)
> C <- c(-2, 3, -2)
> # Titik yang akan diklasifikasikan
> U <- c(1, -3, 5)
> # Fungsi menghitung jarak
> jarak <- function(p, q) {
+   sqrt(sum((p - q)^2))
+ }
> # Hitung jarak
> dA <- jarak(U, A)
> dB <- jarak(U, B)
> dC <- jarak(U, C)
> # Tentukan cluster
> if (dA <= dB && dA <= dC) {
+   cluster <- "A"
+ } else if (dB <= dA && dB <= dC) {
+   cluster <- "B"
+ } else {
+   cluster <- "C"
+ }
> cat("Titik U =", U, "\n")
Titik U = 1 -3 5
> cat("Jarak ke Cluster A =", round(dA, 3), "\n")
Jarak ke Cluster A = 4.583
> cat("Jarak ke Cluster B =", round(dB, 3), "\n")
Jarak ke Cluster B = 1.414
> cat("Jarak ke Cluster C =", round(dC, 3), "\n")
Jarak ke Cluster C = 9.695
> cat("Titik U masuk ke Cluster", cluster, "\n")
Titik U masuk ke Cluster B
> |
```

```
> # Titik pusat cluster
> A <- c(2, 1, 3)
> B <- c(1, -4, 6)
> C <- c(-2, 3, -2)
> # Titik yang akan diklasifikasikan
> U <- c(-1, 3, -1)
> # Fungsi menghitung jarak
> jarak <- function(p, q) {
+   sqrt(sum((p - q)^2))
+ }
> # Hitung jarak
> dA <- jarak(U, A)
> dB <- jarak(U, B)
> dC <- jarak(U, C)
> # Tentukan cluster
> if (dA <= dB && dA <= dC) {
+   cluster <- "A"
+ } else if (dB <= dA && dB <= dC) {
+   cluster <- "B"
+ } else {
+   cluster <- "C"
+ }
> cat("Titik U =", U, "\n")
Titik U = -1 3 -1
> cat("Jarak ke Cluster A =", round(dA, 3), "\n")
Jarak ke Cluster A = 5.385
> cat("Jarak ke Cluster B =", round(dB, 3), "\n")
Jarak ke Cluster B = 10.1
> cat("Jarak ke Cluster C =", round(dC, 3), "\n")
Jarak ke Cluster C = 1.414
> cat("Titik U masuk ke Cluster", cluster, "\n")
Titik U masuk ke Cluster C
> |
```